

Stored-Expert Consolidation (Formulation B)

Config selector: `ppo.method = stored_expert`

Source: `crl/ppo/stored_expert.py` (`StoredExpertTrainer.train, _global_value`)
+ `crl/ppo_continual.py` (`_stored_expert, _load_all_experts,`
`_init_global_from_expert`)

Math: `docs/Objective_for_Continual_Reinforcement_Learning.pdf`,
"When we have the expert models stored", eqs 34-46.

Idea

Every task has a precomputed FROZEN single-task expert. Its value V_i^* is a fixed CEILING (eq 34). With a ceiling per task there is no adversary and nothing to trade, so the min-max collapses (doc Q2): consolidation becomes a plain gap-weighted REGRESSION of the global policy $\pi_G = \pi_\phi$ toward all ceilings at once (eq 37). Past and current tasks are SYMMETRIC. There is NO dual multiplier μ , NO local phase, NO min-max.

Per-task actor coefficient (eq 46, start-distribution / scalar level):

$$\text{coeff}_i = \omega_i * 2 * \max(0, V_i^* - V_i^G)$$

V_i^* = frozen expert Monte-Carlo value; V_i^G = global's FRESH Monte-Carlo value on task i (eq 44). V_i^G is estimated by ROLLOUT, never read off the critic head -- reading the critic head, which the actor update drags along on the shared trunk, was the bug the doc calls out (Q1). Both V_i^* and V_i^G come from the SAME evaluator, so reward scaling / clipping / horizon match, dissolving the scale-mismatch silent bug (Q4). At the ceiling the gap is zero, the coefficient vanishes, the actor stops moving on that task (eqs 41,46) -- the intended fixed point; no constraint is needed (eq 37 is unconstrained).

Inputs

`family` : ordered list of K Atari tasks
`global_policy` : shared-trunk, per-task actor+critic heads
`ppo.expert_dir / expert_refs_path` : frozen Impala experts + cached refs
`ppo.global_iters, constraint_episodes, constraint_every`
`ppo.constraint_greedy, eval_seed, warmstart_head_from_expert`
`ppo.clip_ratio, lr, vf_coef, ent_coef, gae_lambda, n_envs, n_steps`
`seed`

Outputs

`global_policy` after consolidating all K tasks (`final_policy.pt`)
`eval_matrix` : retention matrix, one row per task k (`eval_matrix.json`)

```
expert_refs.json (V*, expert scores)
per-iteration stored_expert / retention / gaps logs
```

Orchestration (stored_expert)

```
games <- names of family.tasks ; K <- len(family)

# 1) load frozen experts + fixed reference ceilings (_load_all_experts)
for i in 0..K-1:
    expert_i <- load frozen single-task Impala expert for game i (grad off)
    if cached expert_refs.json matches games: (V*_i, score_i) <- cache
    else: V*_i <- eval_value_greedy(expert_i, task_i) # eq 44 ceiling
        score_i <- raw greedy score of expert_i
# store _experts, _expert_values (V*), _expert_scores

# 2) init global from the task-1 expert (_init_global_from_expert)
global.trunk <- expert_0.trunk # identical shared trunk
global.actors[0] <- expert_0.actor # actor head 0 seeded
# critic head NOT seeded: not in eq 46's actor gradient, and a copied critic
# is invalid on the shared trunk; PP0's value loss retrains it.
log retention row after T1 ; save checkpoint

# 3) incremental consolidation over tasks 2..K
trainer <- StoredExpertTrainer(...)
for k = 2 .. K:
    tasks_k <- family.tasks[0 .. k-1] # ALL seen tasks (no split)
    expert_vals <- [V*_0 .. V*_{k-1}]
    omega <- [1/k] * k # uniform over ALL k tasks
    if warmstart_head_from_expert:
        global.actors[k-1] <- expert_{k-1}.actor # seed new head so gap starts
                                                    # small (critic left to PP0)
    trainer.train(global, tasks_k, expert_vals, omega,
                  num_iters=global_iters, seed=seed+1000*k, current_task=k)
    log {"phase":"gaps", task=k, V_expert=expert_vals}
    log retention row (global vs expert per seen task) ; save checkpoint

save final_policy
```

StoredExpertTrainer.train (one consolidation phase after task k; eqs 44-46)

```
k <- len(tasks)
optimizer <- Adam(trainable params of global)
collectors[i] <- RolloutCollector(task_i, seed+101+i) for all i in 0..k-1
    # replay-free fresh rollouts in EVERY (past+current) env;
    # Formulation B has no past/current split.
gaps, vg, sg, coeffs <- zero / NaN vectors of length k
```

```

for it = 0 .. num_iters-1:

    # fresh rollout stream per task (each with GAE advantages, doc 03)
    streams <- [collectors[i].collect(global, gae_lambda) for i in 0..k-1]

    # ---- refresh gap coefficients on the SLOW timescale ----
    if it % constraint_every == 0:
        for i in 0..k-1:
            (vg[i], sg[i]) <- _global_value(global, task_i)    # FRESH MC, eq 44
                                # SAME evaluator as V*_i ; NEVER the critic head
            gaps[i] <- max(0, V*_i - vg[i])                    # hinge [g_i]_+, eq 35/36
            coeffs <- [ omega[i] * 2 * gaps[i]  for i in 0..k-1 ] # eq 46
        # between refreshes coeffs are held fixed (mirrors the mu cadence in A)

    # ---- one PPO update over ALL k streams (doc 03) ----
    optimize_batches(global, opt, streams, actor_coeffs = coeffs)
    # inside: coeffs renormalized by their sum; per-stream clipped surrogate,
    # standard value loss + entropy, global grad-norm clip. No dual, no bc.

    probe("stored_expert", k)
    periodically log V_global, V_expert, gap, coeff, ratio_v = V_G / V*

# stopping: num_iters (global_iters) exhausted. NO early-stop threshold, NO
# dual update -- eq 37 is an unconstrained minimization of shortfall.

```

_global_value (the fresh MC estimate, eq 44)

```

(v, s) <- evaluate_value_and_score(policy, task, constraint_episodes, n_envs,
    seed=eval_seed, greedy=constraint_greedy,
    noop_enumerate=eval_noop_enumerate, max_ep_steps=eval_max_ep_steps)
return v    # discounted value V_i^G (raw score s logged only)
# identical settings to how V*_i was computed -> directly comparable (Q4).

```

What is updated, and when

phi (global params) : every iteration (ascends eq 46 gap-weighted PG).
 coeffs (gaps) : every constraint_every iters; held fixed between.
 V*_i (ceilings) : computed ONCE at load; frozen forever (eq 41).
 V_i^G : re-estimated every constraint_every iters, all tasks.
 NO mu / NO adversary / NO local reference at all.

Key intermediate quantities

V*_i = expert_values[i] : frozen expert ceiling (eq 34)
 V_i^G = vg[i] : global's fresh MC value on task i (eq 44), moves

$\text{gap}_i = [V^*_i - V_i^G]_+ :$ one-sided hinge (eq 35/36)
 $\text{coeff}_i = 2 * \omega_i * \text{gap}_i :$ gap-weighted actor coefficient (eq 46); $\rightarrow 0$ at ceiling
 $\omega_i = 1/k :$ uniform weight over ALL k seen tasks

Implicit assumptions

- A frozen expert exists per game (precompute experts as fixed references).
- Global value MUST come from rollouts, not the critic head (Q1 bug).
- Expert value and global value share one evaluator / reward scale (Q4).
- Symmetric treatment: no current-task floor, no min-max -- a well trained expert already bounds each task so the global cannot trade one for another.

Code <-> equation map

ceiling bound $J_k(\pi_g) \leq J^*_{\{1:k\}}$ eq 34
 per-state gap $g_i = V^*_i - V_i^G$ eq 35
 one-sided squared shortfall F_i eq 36
 consolidation objective $\min \sum \omega_i F_i$ eq 37
 state-conditioned policy gradient eqs 38-40
 frozen expert $\Rightarrow \text{grad } V^*_i = 0$ eq 41
 grad of squared hinge (factor 2 $[g]_+$) eqs 42-43
 MC value / gradient estimators eqs 44-45
 gap-weighted global update $\text{coeff}_i = 2 \omega_i [g_i]_+ \dots$ eq 46
 no multiplier update (eq 37 unconstrained) doc text after eq 46